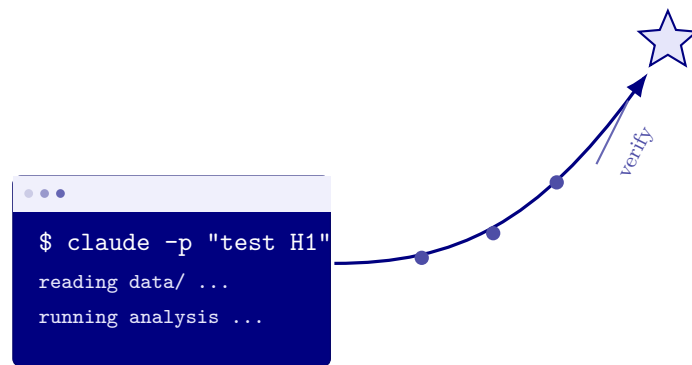


Research at Terminal Velocity

A Hands-On Guide to Agentic AI in Applied Research
with Claude Code, opencode, and Google Antigravity



Ibrahim Abualhaol

2026

Contents

Disclaimer	4
About the Author	5
Preface	6
Introduction	7
I The Case and the Method	8
1 The New Instrument	9
1.1 From Telescope to Terminal	9
1.2 What AI Has Actually Contributed	9
1.3 The Agentic Command Line	10
1.4 What This Book Means by Applied Research	10
2 Critical Thinking Before Tooling	12
2.1 Hypotheses Worth Testing	12
2.2 Can AI Generate the Ideas Themselves?	12
2.3 The Productivity Mirage	13
2.4 Base Rates: What the Reproducibility Crisis Teaches	14
2.5 A Working Epistemic Posture	14
3 A Methodology for AI-Assisted Research	16
3.1 The Eight-Stage Loop	16
3.2 Diverge, Then Converge	16
3.3 Pre-Registration as a Prompt	17
3.4 Provenance: The Agentic Lab Notebook	17
3.5 Scaling the Loop: Solo, Team, Program	18
II The Toolchain	19
4 Anatomy of an Agentic CLI	20
4.1 The Agent Loop	20
4.2 Tools, Permissions, and Blast Radius	20
4.3 Memory Files and the Model Context Protocol	21
4.4 The Four Tools at a Glance	21
5 Claude Code as a Research Assistant	23
5.1 Setup and the Research Contract	23
5.2 The Interactive Session	23
5.3 Extending the Tool: Skills, Subagents, Hooks	24
5.4 Headless Mode: The Agent as a Unix Tool	24
5.5 A Complete Exchange, Annotated	25

6	opencode and the Google Stack	27
6.1	opencode: The Open-Source Agent	27
6.2	Google Antigravity: The Agent-First Platform	27
6.3	Gemini CLI: The Free Workhorse	28
6.4	Choosing: A Working Decision Rule	28
6.5	Budgeting: Tokens, Time, and Attention	29
III	The Practice: Workshop Labs	30
7	Lab 1: Literature Review and Evidence Synthesis	31
7.1	The Shape of the Pipeline	31
7.2	Sweep and Extract	31
7.3	Synthesize: The Claim–Evidence Matrix	32
7.4	The Verification Ritual	32
7.5	Adapting the Lab to Your Field	33
8	Lab 2: From Hypothesis to Experiment	34
8.1	The Hypothesis Card	34
8.2	Executing the Frozen Plan	34
8.3	Robustness: Attacking Your Own Result	35
9	Lab 3: Reproducibility and Reporting	37
9.1	The Reproducibility Stack	37
9.2	Reporting: The Agent as Critic, Never Author	38
10	The Researcher’s Field Manual	40
10.1	The Validation Hierarchy	40
10.2	The Prompt Library	40
10.3	The Checklists	41
10.4	Situations to Rules	41
10.5	Objections, Answered Briefly	42
10.6	Start Monday	42
A	Workshop Setup Guide	43
A.1	Prerequisites	43
A.2	Installing the Tools	43
A.3	The Practice Repository	44
A.4	Seat Check	44
B	Templates	45
B.1	The Contract File	45
B.2	The Hypothesis Card	45
B.3	The Decisions File	46
B.4	The Reproduction Makefile	46
	Further Reading: A Map of the Evidence	47
	References	48

Disclaimer

This book is a practical guide to using agentic AI tools in applied research. It describes specific software products — Claude Code, opencode, Google Antigravity, and Gemini CLI — as they existed at the time of writing; commands, features, pricing, and availability change quickly, and readers should consult each tool’s current documentation before relying on details. The empirical studies cited here are summarized in good faith, with their limitations noted, but this book is not a substitute for reading the primary sources — indeed, one of its central lessons is that you should. Nothing here is legal, ethical-review, or data-governance advice: institutional policies on AI use, data handling, and authorship differ, and they take precedence over any workflow suggested in these pages.

About the Author

Ibrahim Abualhaol is an AI Technical Lead and Senior Consultant in Data Science at Dell Technologies' Office of the CTO in Ottawa, Canada. He holds a Ph.D. in Electrical and Computer Engineering along with an M.Eng. in Technology Innovation Management, and is a Professional Engineer (P.Eng.) in Ontario and a Senior Member of IEEE (SMIEEE). His research spans machine learning, real-time big-data analytics, cybersecurity, and wireless communications, reflected in 36+ patents, over 1,000 citations, and 14 journal publications. Prior to his current role, he held academic appointments at Khalifa University (UAE) and Princess Sumaya University (Jordan).

Preface

Applied research has always advanced by instrument. The telescope did not merely speed up astronomy; it changed what counted as an observable question. The computer did not merely speed up calculation; it made simulation a third mode of science alongside theory and experiment. Agentic AI — software that can read a codebase, run commands, browse sources, and iterate toward a goal with limited supervision — is the newest instrument, and like its predecessors it changes not just the speed of research but its shape.

This book was written for a specific, practical situation: a researcher or engineer sitting down at a terminal with a real question, a real dataset or system, and one of the new agentic command-line tools installed. It grew out of workshop material, and it keeps a workshop's temperament. Every claim about what these tools do well or badly is tied to a citable study or a runnable example, and the second half of the book is organized as laboratories you can execute rather than chapters you merely read.

Two convictions run through it. The first is that the instrument amplifies the method: an agent in the hands of a careful researcher compounds care, and in the hands of a careless one compounds carelessness, faster than any previous tool. That is why the book insists on critical thinking about ideas and hypotheses *before* touching the toolchain, and on validation rituals after. The second is that tool literacy is now part of research methodology. Knowing when to reach for Claude Code, when an open-source agent like `opencode` fits better, and what an agent-first platform like Google Antigravity adds is as much a methods question as choosing between a t-test and a permutation test.

Read Part I before the labs; it earns the workflows the labs assume. Part II can be skimmed by tool-experienced readers and studied by newcomers. Part III is meant to be run, not read — ideally with a terminal open beside the page.

Introduction

This book answers one question: *how do you use agentic AI to do applied research that is faster and also more rigorous — not faster and sloppier?* The answer unfolds across three parts.

Part I: The Case and the Method (Chapters 1–3) builds the foundation. Chapter 1 surveys what AI systems have actually contributed to science — from protein structures to new mathematics to autonomous chemistry — and locates the agentic command line within that landscape. Chapter 2 is the critical-thinking chapter: it examines the evidence on AI-generated research ideas, the documented gap between fluency and validity, the sobering randomized-trial results on developer productivity, and what the reproducibility crisis teaches about trusting exciting results, including your own. Chapter 3 assembles the methodology: an eight-stage loop from question to report, with explicit human verification gates, that the rest of the book instantiates.

Part II: The Toolchain (Chapters 4–6) covers the instruments. Chapter 4 dissects the anatomy shared by all agentic CLI tools — the agent loop, tools and permissions, memory files, and the Model Context Protocol — so that skills transfer across products. Chapter 5 is a deep dive on Claude Code as a research assistant: project memory, plan mode, skills, subagents, hooks, and headless pipelines. Chapter 6 covers the alternatives and complements: the open-source, provider-agnostic *opencode*; Google’s agent-first *Antigravity* platform; and *Gemini CLI* — and gives concrete rules for choosing among them.

Part III: The Practice (Chapters 7–10) is the workshop. Three hands-on laboratories walk through the research lifecycle: Lab 1 (Chapter 7) builds a literature-review and evidence-synthesis pipeline; Lab 2 (Chapter 8) takes a hypothesis through a pre-registered, agent-assisted analysis with robustness checks; Lab 3 (Chapter 9) hardens the result into a reproducible, reportable artifact. Chapter 10 closes with the field manual — the prompt library, checklists, and quick-reference tables designed to be used long after the workshop ends.

The through-line is a single principle: *let the agent carry the work; keep the judgment.* Delegation without verification is not research acceleration — it is error acceleration.

Part I

The Case and the Method

Chapter 1

The New Instrument

Every era of research is shaped by its dominant instrument. The seventeenth century had the telescope, the twentieth the computer, and the last decade the large-scale learned model. Each changed not only how fast existing questions could be answered but which questions were worth asking at all. This chapter makes the case that agentic AI — and specifically the agentic *command line* — is the newest such instrument for applied research, surveys what AI systems have already contributed to science so the claim rests on evidence rather than enthusiasm, and fixes the vocabulary the rest of the book uses.

1.1 From Telescope to Terminal

An instrument, in the research sense, is anything that extends what a trained mind can observe or manipulate. The interesting historical pattern is that instruments do not distribute their benefits evenly: they amplify the methodology of whoever holds them. Galileo pointed the telescope at Jupiter with a hypothesis in hand; his contemporaries pointed identical tubes at the sky and saw smudges. The computer accelerated both careful numerical analysis and the mass production of unexamined statistical noise. There is no reason to expect agentic AI to be different, and the evidence reviewed in Chapter 2 confirms that it is not: the same tools that produce genuine acceleration in some hands produce confident error in others.

This book therefore treats tool adoption and research methodology as a single subject. The labs in Part III teach commands and prompts, but every command is embedded in a verification discipline, because the instrument is only as good as the method it amplifies.

1.2 What AI Has Actually Contributed

Skepticism about AI hype is warranted, so it is worth being precise about what has already happened — not demos, but contributions that survived peer review or independent replication.

The clearest case is structure prediction. AlphaFold 2 reached near-experimental accuracy on protein structures in the CASP14 assessment, and its predictions are now ordinary working material in structural biology [13]. In mathematics, FunSearch paired a language model with an evaluator loop and found new constructions for the cap set problem — the first genuinely new mathematical discovery from an LLM-driven search, published with proofs that stand independent of how they were found [19]. In chemistry, the Coscientist system planned and physically executed multi-step syntheses, including correctly operating a liquid-handling robot from documentation it read itself [6]. A broad review of the pattern — AI as hypothesis generator, experiment designer, and data interpreter across disciplines — is given by Wang et al. [21].

The frontier has since moved from AI *inside* the research pipeline to AI *running* the pipeline. The AI Scientist framework generates ideas, writes and runs code, and drafts papers end-to-end [15]; its successor produced a manuscript that passed workshop-level peer review [22]. Google’s AI co-scientist takes a different route — a multi-agent tournament that generates, critiques, and ranks hypotheses for human scientists, with early wet-lab validations in drug repurposing and microbiology [10]. Figure 1.1 lays out the sequence.

	2021	2022	2023	2024	2025	2026
Prediction	AlphaFold 2					
Search & proof			FunSearch			
Autonomous lab			Coscientist		AI Sci. v2	
Agentic assistant				AI Scientist	co-scientist	agentic CLIs & agent IDEs

Direction of travel: from models *inside* the pipeline to agents *running* it

Figure 1.1: Milestones in AI-assisted research, 2021–2026. Rows are modes of contribution; filled cells mark representative published systems [6, 10, 13, 15, 19]. The trend runs diagonally: each year the AI takes responsibility for a larger fraction of the research loop, ending (bottom right) at the general-purpose agentic tools this book teaches.

Three honest caveats belong next to this record. First, the celebrated results cluster in domains with fast, cheap, automated verification — a protein structure can be scored, a cap set construction checked, a synthesis assayed. Applied research whose validation is slow or human (user studies, field trials, policy) inherits less of the speedup. Second, several headline systems required heavy human curation behind the scenes; the papers say so, and secondary reporting usually does not. Third, none of this removes the researcher’s accountability for what gets published under their name — a theme Chapter 2 develops with less comfortable evidence.

1.3 The Agentic Command Line

The systems above are bespoke. What changed for working researchers in 2025–2026 is that the same underlying capability became available as general-purpose tools that live where research computing actually happens: the terminal.

Agentic AI

An AI system that pursues a stated goal through an iterative loop of its own actions: it plans, invokes tools (file edits, shell commands, web requests), observes the results, and revises its plan — rather than returning a single response to a single prompt. The defining property is *closed-loop tool use with limited supervision*, not any particular model or interface.

Claude Code, opencode, and Gemini CLI are agentic AI in exactly this sense, packaged as command-line programs; Google Antigravity wraps the same loop in an agent-first development platform [1, 2, 8, 9]. Chapter 4 dissects their shared anatomy. What matters here is why the *terminal* form factor fits research work specifically. Research computing is already file- and script-shaped: datasets in directories, analysis in version-controlled code, results in figures and logs. A terminal agent operates on that substrate directly — it can read your data-loading code, run your tests, re-execute your pipeline, and leave an auditable trail in the shell history and git log. The chat window asks you to carry context to the model; the terminal agent comes to where the context already lives.

1.4 What This Book Means by Applied Research

Applied research, throughout this book, means investigation with a decision at the end of it: an engineering team choosing an architecture, a data scientist testing whether an intervention moved a metric, a graduate student validating a method against baselines, an analyst quantifying a risk. It shares the standards of basic science — falsifiable claims, controlled comparison,

reproducible analysis — but its output is a defensible decision, which raises the cost of being confidently wrong and lowers the tolerance for unverified fluency.

What the rest of this book assumes

- You have a terminal, git, and Python (or an equivalent analysis stack), and are comfortable with both.
- You have access to at least one agentic CLI tool — the labs use Claude Code as the running example and note the opencode and Gemini CLI equivalents.
- Your data and your institution’s AI-use policies permit sending working material to a model provider; if not, Chapter 6 discusses open-source and local-model options.
- You are accountable for your outputs. No workflow in this book delegates that.

The next chapter turns to the uncomfortable part of the evidence: what happens to ideas, productivity, and judgment when researchers adopt these tools without a method — and why critical thinking has to be installed before the toolchain, not after.

Chapter 2

Critical Thinking Before Tooling

The fastest way to misuse an agentic tool is to hand it a bad question. This chapter is the book’s critical-thinking core: it establishes what makes an idea worth testing, examines what the empirical record actually says about AI-generated research ideas and AI-assisted productivity — including results that cut against the marketing — and imports the hardest-won lessons of the reproducibility crisis into the agentic era. Everything in the methodology of Chapter 3 is a response to a failure mode documented here.

2.1 Hypotheses Worth Testing

A research idea and a hypothesis are different objects. An idea is a direction (“agentic tools probably help with literature reviews”); a hypothesis is a claim about the world specific enough to be wrong. The discipline of converting one into the other is the single highest-leverage step in applied research, because every downstream hour — human or agent — is spent inside the frame it fixes.

Hypothesis

A falsifiable claim relating defined variables in a defined population or system, stated with the direction and rough magnitude of the expected effect, such that a feasible observation could refute it. “Tool *X* improves productivity” is an idea; “developers using tool *X* complete our issue-fix benchmark at least 20% faster than matched controls, without increasing post-merge defect rate” is a hypothesis.

Four properties separate testable hypotheses from decorated ideas, and Table 2.1 turns them into the rubric used in every lab in Part III. *Falsifiable*: some achievable observation would refute it. *Mechanistic*: it says why the effect should exist, which is what lets a result generalize beyond the exact setting measured. *Decision-relevant*: some real choice depends on the answer — otherwise the work is trivia with error bars. *Scoped*: population, conditions, and metrics are pinned down before data arrives, not after.

Table 2.1: The hypothesis-quality rubric. Score each 0–2; anything scoring 0 on falsifiability or scope is returned for reformulation before any agent time is spent on it.

Property	Test question	Failure smell
Falsifiable	What result would refute this?	“Any result confirms it”
Mechanistic	Why would this be true?	Correlation with a story
Decision-relevant	Who acts differently on the answer?	“Interesting to know”
Scoped	Population, metric, threshold fixed?	Moving targets

2.2 Can AI Generate the Ideas Themselves?

It is tempting to delegate ideation too, and here the evidence is usefully two-sided. The largest controlled comparison to date recruited over 100 NLP researchers to blind-review ideas written

by experts and by an LLM ideation agent: the AI-generated ideas were rated significantly *more novel* than the human ones, but somewhat weaker on feasibility [20]. A follow-up phase of the same program had researchers actually execute a sample of the ideas, and the novelty advantage shrank on contact with implementation — exciting-sounding ideas were disproportionately the ones that failed to survive being built.

That asymmetry — machines strong on divergence, weak on the feasibility judgment that requires knowing how work actually fails — is exactly what a good workflow can exploit: use the model as a high-volume idea generator and the human as a ruthless filter, which is how Chapter 3’s diverge–converge stage is built, and how Google’s co-scientist tournament is architected internally [10].

Why fluency is not validity

Language models are optimized to produce text that reads as competent; whether a claim is *true* is only correlated with whether it *reads well*, and the correlation weakens exactly where research lives — at the frontier where surface plausibility and truth come apart. This is why every fluent output in this book’s workflows is treated as a *claim awaiting verification*, and why the verification is always run against something other than the model’s own confidence: a primary source, a rerun, a held-out test. Anthropologists of science Messeri and Crockett give the systemic version of the warning: AI tools invite “illusions of understanding” in which researchers produce more while comprehending less, mistaking the tool’s fluency for their own insight [16].

Messeri and Crockett [16] identify a further risk worth naming before the toolchain chapters: *monocultures of knowing*. When many researchers route their thinking through the same few models, the questions asked and the framings explored quietly converge. The practical counter-measure adopted in Part III is cheap: force the agent to argue multiple sides (critic prompts), and keep at least one deliberately AI-free reasoning pass on anything that matters.

2.3 The Productivity Mirage

The productivity evidence deserves the same scrutiny, because it is routinely summarized as more settled than it is. Figure 2.1 compresses the four studies every workshop participant should know.

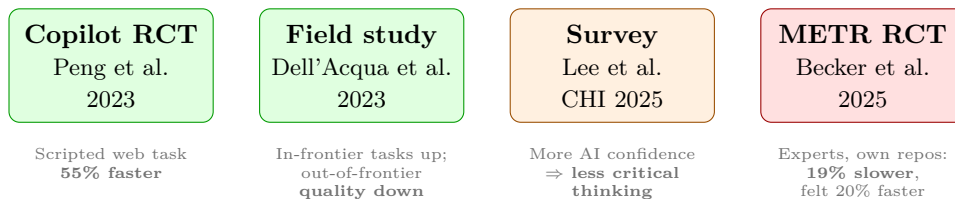


Figure 2.1: Four load-bearing studies on AI-assisted knowledge work, ordered from most to least favorable. Green cards report gains, the red card a measured loss. The pattern is not contradiction but moderation: task familiarity, task frontier, and user calibration determine the sign of the effect [5, 7, 14, 18].

Read together, the studies say the effect of AI assistance is *conditional*. On well-specified, greenfield tasks, a controlled experiment found a 55% speedup [18]. On a portfolio of consulting tasks, gains were large inside the model’s “jagged frontier” of competence and turned into *quality losses* just outside it — with participants unable to feel where the frontier was [7]. A survey of knowledge workers found that higher confidence in the AI predicted *less* critical thinking about its outputs [14]. And the most rigorous test in the most research-like conditions is the most sobering:

The expert slowdown result

METR ran a randomized controlled trial with experienced open-source maintainers working on their *own* mature repositories with early-2025 tools. Allowing AI made them 19% *slower* — while they self-estimated a 20% speedup, and continued to believe in the speedup after the trial [5]. The conditions — deep expertise, high-context codebases, strict quality bars — resemble applied research more than any other study’s do. The lesson is not “agents don’t work”; it is that *perceived* speedup is inadmissible evidence. If your workflow matters, measure it, on your own tasks, before scaling it.

2.4 Base Rates: What the Reproducibility Crisis Teaches

Applied research had a trust problem before AI arrived, and its arithmetic transfers directly. Ioannidis [11] showed formally that when tested hypotheses have low prior odds and analyses have many degrees of freedom, most published positive findings will be false — not through fraud, but through selection. Empirically, a coordinated replication of 100 psychology studies succeeded in under half of them [17], and in a survey of 1,576 scientists, over 70% had failed to reproduce another lab’s experiment and half had failed to reproduce their own [3].

Agentic tools plug into this machinery as an amplifier. An agent can try analysis variants at a rate no human could, which multiplies the researcher’s degrees of freedom — the exact quantity Ioannidis’s model punishes. It never tires of searching for a specification in which the effect appears. And it produces polished prose and figures that make a fragile result *look* finished, lowering the odds that anyone, including its author, gives it the skeptical second pass. The same loop that accelerates honest analysis accelerates p-hacking if pointed even slightly askew.

The response is not abstinence but structure: decide the analysis before seeing results, log everything the agent tried (not just what worked), and subject your own exciting findings to the adversarial review you would give a rival’s. Chapter 3 builds that structure; Labs 2 and 3 make it muscle memory.

2.5 A Working Epistemic Posture

The chapter’s evidence compresses into a posture — four habits of weighing claims that the rest of the book assumes and the field manual (Chapter 10) restates as checklists.

Grade the claim by its verification, not its source. An agent’s output, a preprint’s abstract, and your own recollection sit at the same level until checked; what promotes a claim is the check that has been run against it — a fetched primary source, an independent recomputation, a replication. This is why the labs treat “verified” as a property attached to artifacts, never to tools.

Weight novelty as a cost, not only a virtue. Base-rate reasoning says surprising findings are more often wrong [11], and the ideation evidence says AI-generated ideas skew novel-but-infeasible [20]. The rational response to an exciting result — yours especially — is to raise the verification bar exactly when enthusiasm is telling you to lower it.

Track your calibration, not your impressions. The METR participants were not careless; they were experts whose felt experience pointed the wrong way [5]. The only defense anyone has demonstrated is measurement: keep predictions (“this workflow saves me time,” “this result will replicate”) somewhere they can be scored, and score them.

Keep one line of reasoning model-free. The monoculture risk [16] is not answered by better prompts; it is answered by preserving an independent path to the answer — a back-of-envelope estimate, a hand-derived expectation, a colleague who has not seen the agent’s framing — so that agreement between your reasoning and the model’s is evidence rather than an echo.

None of this is anti-tool; it is the posture that makes heavy tool use safe. The next chapter turns it from posture into procedure.

Chapter 3

A Methodology for AI-Assisted Research

Chapters 1 and 2 established a tension: the instrument is genuinely powerful, and it amplifies whatever method it is embedded in — including no method. This chapter resolves the tension by supplying the method: an eight-stage loop from question to report, with the human effort concentrated at three verification gates. It is deliberately tool-agnostic; Part II supplies the tools and Part III runs the loop end-to-end on real tasks.

3.1 The Eight-Stage Loop

Figure 3.1 shows the whole methodology. Its shape encodes the two design principles argued for in Chapter 2: machines diverge, humans converge (stages 2–3), and analysis decisions precede data contact (stage 5 before stage 6).

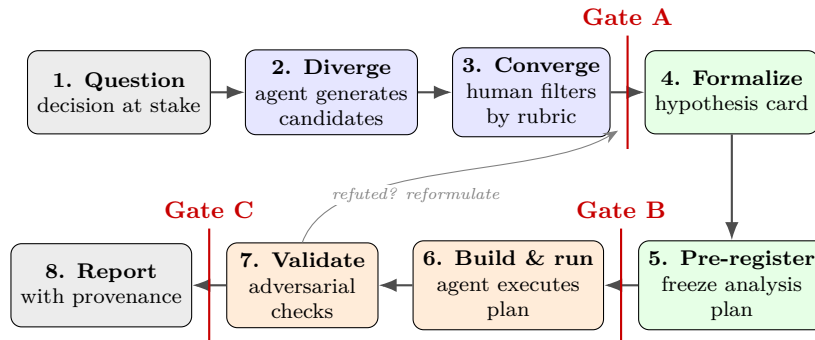


Figure 3.1: The eight-stage loop for AI-assisted applied research. Blue stages are agent-heavy, green stages are human-heavy, orange stages are agent-executed under a frozen plan. The three red gates are human sign-offs that no agent output crosses on its own: Gate A admits a hypothesis to testing, Gate B admits an analysis plan to data, Gate C admits a result to reporting.

The loop is a cycle, not a line: a hypothesis refuted at stage 7 returns to stage 4 cheaply, which is the point. The methodology optimizes for making refutation fast and unembarrassing, because in applied settings a quickly killed idea is a profit, not a failure.

3.2 Diverge, Then Converge

Stage 2 exploits the one thing Chapter 2’s evidence says models do better than experts: producing many novel candidate framings cheaply [20]. The working pattern is volume with structure — ask the agent for fifteen candidate hypotheses about the question, each with the expected mechanism, the observation that would refute it, and its biggest feasibility risk. Then, and this is the step that separates the workflow from vibes, *the human filters with the rubric of Table 2.1*, scoring falsifiability and scope personally rather than asking the model to grade its own suggestions. Google’s co-scientist system embeds the same shape internally — generator agents produce hypotheses, a tournament of critic agents ranks them, and humans judge the

survivors [10] — but you do not need a bespoke multi-agent system; you need one agent run twice with different instructions and one honest reader.

Two practical rules sharpen the converge step. First, require the agent to argue *against* each surviving candidate before you score it (a critic pass); Chapter 2’s monoculture warning is answered by forcing disagreement into the transcript [16]. Second, cap survivors at one to three. A workshop participant who leaves stage 3 with seven hypotheses has not converged; they have deferred the decision to their future, busier self.

3.3 Pre-Registration as a Prompt

Pre-registration

Committing to the hypothesis, primary metric, statistical test, sample or data window, and exclusion rules *before* contact with outcome data, in a form that cannot be silently revised — in this book’s workflows, a version-controlled file the agent is instructed to treat as read-only.

Pre-registration is the reproducibility movement’s most direct answer to the selection machinery of Ioannidis [11], and it translates into agentic practice almost verbatim: the pre-registration *is* a prompt. Lab 2 writes a `hypothesis-card.md` containing the frozen plan and starts the analysis session by loading it into the agent’s context with the instruction “execute this plan; if any step is impossible, stop and report rather than substituting.” The agent’s tirelessness now works for rigor instead of against it: it will follow the frozen plan exactly as happily as it would have explored specifications until something glittered.

Exploration is not forbidden — it is *labeled*. Anything run after seeing outcomes goes in a separate session logged as exploratory, and may feed the next loop iteration’s stage 2, never the current report’s headline claim.

The three gates, as checklists

Gate A — admit to testing: rubric scored by a human; critic pass read; one to three survivors; feasibility risk named.

Gate B — admit to data: hypothesis card committed to git; metric, test, and exclusions frozen; power or sample-size sanity check done; agent instructions reference the card as read-only.

Gate C — admit to reporting: validation stage ran adversarial checks (robustness, reruns, held-out data); provenance ledger complete; at least one claim survived an attempt to kill it; the human can explain the result without opening the transcript.

3.4 Provenance: The Agentic Lab Notebook

The classical lab notebook records what was done so that the work can be audited and repeated. In agent-assisted work most actions are taken by the tool, so the notebook must capture the agent’s activity, not the human’s memory of it. Three artifacts, all cheap, constitute the provenance ledger used throughout Part III: the *session transcript* (every tool covered in Part II can persist or resume its sessions), the *git history* (each loop stage ends in a commit, so the diff between commits is the auditable record of what changed and when), and a *decisions file* — five lines per session, human-written, saying what was decided and why. The transcript answers “what did the agent do,” the diff answers “what changed,” and the decisions file answers the only question the other two cannot: “what did the human choose.”

Design of the collaboration matters as much as logging of it. The strongest causal evidence available says that *how* the assistance is shaped flips its sign: students given raw model answers performed worse once the tool was removed, while students given a tutor-configured version — hints, checks, no final answers — kept the gains [4]. The gates and the frozen-plan pattern are this book’s tutor configuration for research: they position the agent where it compounds your method instead of replacing it.

3.5 Scaling the Loop: Solo, Team, Program

The loop as drawn is a solo discipline, and a single researcher can run it with nothing but the templates in Appendix B. Two adjustments matter as the unit of work grows.

In a *team*, the gates become social rather than personal: Gate A is a ten-minute standup where survivors of the converge stage are defended aloud; Gate B requires the hypothesis card to be approved by someone who did not write it; Gate C’s adversarial review is done by a colleague’s agent under a different contract, which breaks the subtle coupling between an author’s phrasing and their own model’s critique. The provenance ledger also changes audience — the decisions file stops being a memory aid and becomes the interface by which teammates resume each other’s work, so entries earn full sentences.

At *program* scale — many projects, shared infrastructure — the leverage moves into standardization: one contract template, one card format, and one reproduction Makefile across projects, enforced gently by starting every new repository from the same skeleton (Appendix A builds it). The payoff compounds: agents are better at following conventions than people are, so a program whose projects share structure gets cheaper agent onboarding on every project after the first, and cross-project audits (“rerun every active project’s `make verify`”) become a nightly headless job instead of a quarterly ordeal.

What does not change with scale is the location of judgment. Gates can be staffed differently, but each one is still a named human signing a specific artifact; a gate that “the team” owns is a gate nobody owns. Groups that blur this rediscover, at team speed, the individual failure modes of Chapter 2.

With the method in hand, the remaining question is instrumental: which tool, configured how. Part II answers it.

Part II

The Toolchain

Chapter 4

Anatomy of an Agentic CLI

Products differ; the anatomy underneath them barely does. This chapter dissects the machinery shared by Claude Code, opencode, Gemini CLI, and the agents inside Google Antigravity, so that what you learn in one tool transfers to the others — and so that when a tool misbehaves, you can reason about *which component* failed instead of concluding that “the AI is bad today.”

4.1 The Agent Loop

Strip away branding and every agentic CLI runs the same cycle, shown in Figure 4.1: assemble context, call the model, execute whatever tool the model requested, feed the result back, repeat until the model declares the task done or the human intervenes.

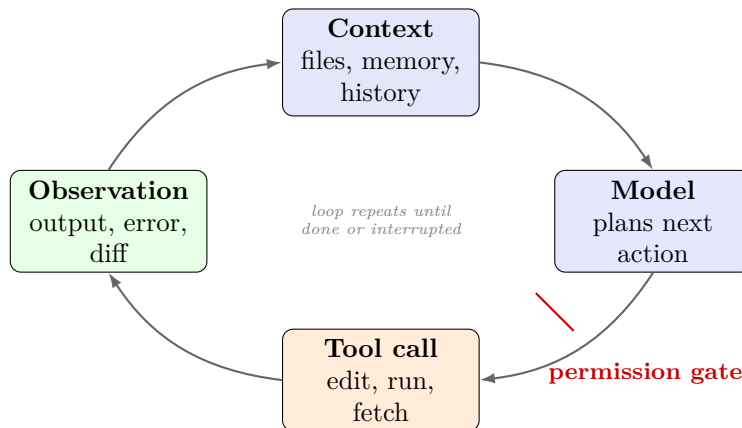


Figure 4.1: The agent loop common to all four tools. The model never touches your system directly: every action passes through a tool call that the harness executes — and, for consequential actions, through a human permission gate. Diagnosing a failure means asking which arc broke: missing context, wrong plan, failed tool, or misread observation.

Each arc of the loop is a distinct engineering concern with a distinct failure smell. *Context assembly* decides what the model can see — when an agent confidently edits the wrong file, the usual cause is that the right one never entered context. *Tool execution* is where the agent meets reality; a tool call that fails loudly is a gift, one that fails silently is how bad results are born. *Observation feedback* closes the loop and is what separates an agent from an autocomplete: the model reads the test failure it just caused and revises. Capability benchmarks measure exactly this closed-loop competence — SWE-bench, the standard yardstick, scores whether an agent can resolve real GitHub issues end-to-end against the repository’s own tests [12].

4.2 Tools, Permissions, and Blast Radius

An agent is exactly as dangerous as the tools it may call. All four products therefore ship a permission system: read-only operations flow freely, while writes, shell commands, and network access require approval — per call, per session, or per rule. Research use argues for a specific discipline. Data directories should be readable but not writable (analyses read raw data;

nothing rewrites it), generated outputs belong in a separate tree, and anything irreversible — pushing, publishing, deleting, emailing — stays behind an explicit human approval, which is the operational form of Chapter 3’s Gate C.

Why terminal-first fits research

A GUI assistant shows you what it did; a terminal agent’s actions *are* shell commands and file diffs, the same primitives your git history and lab notebook already record. That makes provenance (Chapter 3) nearly free: the transcript is a log, every change is a diff, and the whole session replays from the repository state. The terminal is also where research computing already lives — data pipelines, cluster jobs, LaTeX builds — so the agent joins your environment rather than asking your work to move into its own.

4.3 Memory Files and the Model Context Protocol

Two extension mechanisms matter enough to learn before choosing any product, because all four implement both.

Memory files. Each tool reads a markdown file from the project root at session start — `CLAUDE.md` for Claude Code, `AGENTS.md` for opencode, `GEMINI.md` for Gemini CLI [1, 2, 9]. This file is standing instructions: coding conventions, commands that work, and — the research use — the methodological contract. Lab 2 puts the frozen analysis plan and the “stop rather than substitute” rule here, which is how pre-registration survives across sessions without being re-prompted.

Model Context Protocol (MCP)

An open standard for connecting agents to external tools and data sources. An MCP *server* exposes capabilities (query a database, search a literature index, drive an instrument) in a schema that any MCP-capable *client* — including all four tools in this book — can discover and call. For researchers it is the extension point for domain infrastructure: one MCP server for your instrument scheduler, LIMS, or internal data warehouse serves every agentic tool your group uses.

Headless operation. Every tool also runs without its interactive interface — `claude -p "..."`, `opencode run`, or Gemini CLI’s headless scripting mode — reading a prompt from the command line and printing the result to stdout. This turns the agent into a Unix component you can pipe, loop, and schedule, and it is the mechanism behind Lab 1’s literature pipeline.

4.4 The Four Tools at a Glance

Table 4.1 compresses the product landscape as of mid-2026; Chapters 5 and 6 add depth and selection rules.

One reading of the table matters for methodology: the tools are converging on the anatomy while competing on ergonomics. That means the durable investment is in your method — memory-file contracts, gate checklists, provenance habits — which transfer across products, not in any product’s keybindings. The next two chapters tour the products with exactly that bias.

Table 4.1: The toolchain at a glance (mid-2026). All four share the loop, permissions, memory files, MCP, and headless operation; they differ in interface philosophy, openness, and model access [1, 2, 8, 9].

Tool	Form factor		Openness / models	Distinctive strength
Claude Code	terminal (+IDE, desktop)	CLI web,	proprietary; Claude models	skills, subagents, hooks; deepest workflow customization
opencode	terminal (+desktop)	TUI	open source; any provider	provider-agnostic, local models, inspectable
Gemini CLI	terminal CLI		Apache-2.0; Gemini	generous free tier; built-in search grounding
Antigravity	agent-first platform	IDE	proprietary; Gemini 3 + Claude + GPT-OSS	manager for parallel agents; artifacts; browser verification

Chapter 5

Claude Code as a Research Assistant

Claude Code is Anthropic’s agentic tool: a full agent loop in the terminal, with the deepest customization surface of the four products — project memory, plan mode, skills, subagents, hooks, and a headless mode designed for pipelines [2]. This chapter configures it as a research assistant that enforces, rather than merely tolerates, the methodology of Chapter 3. Everything here is used again in the labs.

5.1 Setup and the Research Contract

Installation is one command (`curl -fsSL https://claude.ai/install.sh | bash` on macOS/Linux/WSL, an equivalent PowerShell one-liner on Windows), after which `claude` launched inside a project directory starts a session with that directory as its world. The first research artifact to create is not a prompt but the project’s `CLAUDE.md` — read automatically at every session start, which makes it the natural home for the methodological contract:

```
# CLAUDE.md -- research contract for this project

## Project
Effect of retrieval augmentation on support-ticket
triage accuracy. Pre-registration: analysis/PLAN.md.

## Hard rules
- analysis/PLAN.md is READ-ONLY. Execute it exactly.
  If a step is impossible, STOP and report -- never
  substitute a different test or metric.
- data/raw/ is read-only. Write outputs to results/.
- Every claim in a summary must name the script and
  input that produced it.
- Label any analysis not in PLAN.md as EXPLORATORY
  in all output.

## Environment
- 'make test' runs the analysis unit tests.
- Python env: 'source .venv/bin/activate'.
```

Listing 5.1: A research-project `CLAUDE.md`: standing instructions every session inherits. The contract encodes Chapter 3’s gates as tool behavior.

The contract does real work. Session-to-session, it is what makes pre-registration durable: you cannot forget to re-state the frozen plan, because the tool reads it before you type anything.

5.2 The Interactive Session

A research session has a rhythm worth making explicit, because it maps onto the agent loop of Chapter 4. Figure 5.1 traces a representative exchange from Lab 2’s exploratory phase.

Plan mode — entered with a keystroke — is the feature to teach first: it lets the agent explore the repository and propose an approach while being mechanically prevented from changing anything. The approval that exits plan mode is a natural checkpoint at which to ask Chapter 2’s

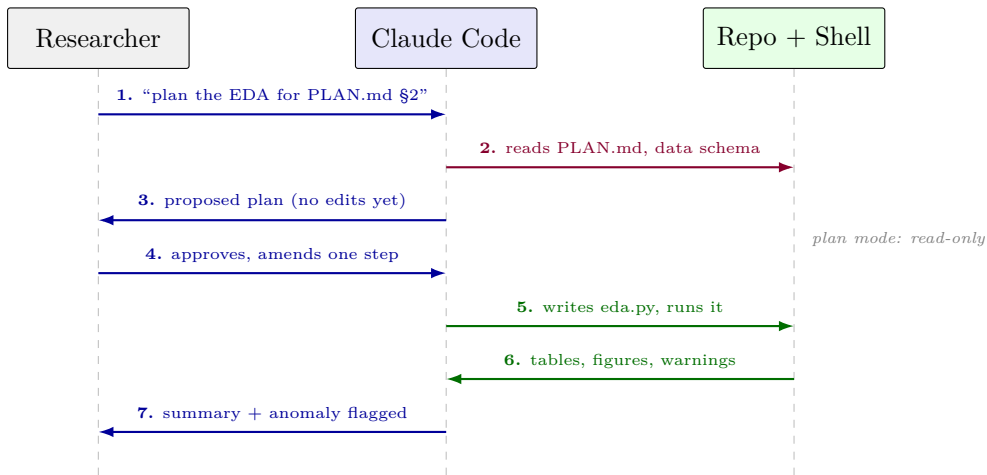


Figure 5.1: A research session as a sequence diagram. Steps 1–3 happen in plan mode, where the agent may read but not modify; the researcher’s approval at step 4 is a miniature Gate B. Only then does the agent write and execute code (steps 5–6), and its summary at step 7 must trace claims back to artifacts per the `CLAUDE.md` contract.

questions: is the proposed analysis the pre-registered one, and would I accept this plan from a new research assistant?

Worked micro-example: interrogating a surprise

During an EDA session, the agent reports that removing one customer segment flips a key correlation’s sign. The tempting move is to celebrate a finding. The contract move is: (1) ask the agent for the exact filter and row counts behind both numbers; (2) have it write a five-line script that reproduces the flip from raw data; (3) run the script yourself; (4) log the anomaly in the decisions file and *park it* — it is exploratory, so it feeds the next loop’s hypothesis stage, not this report. The agent did the labor; the judgment stayed home.

5.3 Extending the Tool: Skills, Subagents, Hooks

Three mechanisms turn one-off prompting into reusable lab infrastructure. *Skills* are markdown-packaged procedures the agent loads on demand — a lab can encode its own SOPs (“run the ablation protocol,” “build the results notebook”) as skills, and this book itself was typeset by one. *Subagents* are scoped delegate sessions with their own context and restricted tools: a reviewer subagent that can read but not write is the clean way to run Chapter 3’s critic passes without the critic contaminating the builder’s context. *Hooks* run shell commands on tool events — the research use is provenance enforcement, e.g. a post-edit hook that appends every file change to the session ledger, making the lab notebook self-writing.

5.4 Headless Mode: The Agent as a Unix Tool

The flag `-p` (print) runs one prompt non-interactively, which makes the agent composable with everything else in your toolbox:

```
# Triage today's experiment logs for anomalies
tail -500 logs/run-042.log | \
  claude -p "list anomalies with line numbers; JSON"

# Batch-extract structured fields from papers (Lab 1)
```

```

for f in corpus/*.txt; do
  claude -p "Extract {claim, method, N, effect size} \
    as JSON from: $(cat "$f")" >> extracted.jsonl
done

# Nightly: check that results still reproduce
claude -p "run make reproduce; report any metric that \
  moved by >1% vs results/baseline.json" >> ci-repro.log

```

Listing 5.2: Headless invocations: the agent as a pipeline stage. Each command reads stdin or files, does one bounded job, and writes stdout — Unix rules apply.

Claude Code habits that pay off in research

- Put the methodological contract in `CLAUDE.md`; treat prompt-repetition as a smell that something belongs there.
- Start every consequential task in plan mode; approve plans, not diffs you haven't read.
- Use a read-only reviewer subagent for critic passes; never let the builder grade its own work.
- Reserve `-p` for bounded, verifiable jobs — extraction, checking, triage — and keep open-ended reasoning interactive where you can steer.
- Commit at every stage boundary so the git log is your provenance ledger.

5.5 A Complete Exchange, Annotated

Prompt craft is best taught by contrast. Below is the same request made twice — the version participants type on day one, and the version the methodology produces. Both are real workshop artifacts.

```

-- Day one --
Analyze the ticket data and tell me if retrieval
helps.

-- Contract-shaped --
Stage 6 session (build & run) for H1.
Load analysis/PLAN.md and restate it in three lines;
wait for my OK before writing code.
Then execute exactly: exclusions (log counts), top-1
accuracy for both systems, permutation test, bootstrap
CI, figures to results/fig/.
Print every filter's row count as you go. If any step
is impossible, stop and say why. Nothing beyond the
plan; anything else you notice goes to a NOTES section
labeled EXPLORATORY.

```

Listing 5.3: The day-one prompt and the contract-shaped prompt for the same task. The second is longer to type and dramatically cheaper to verify.

The differences are Chapter 3 made operational. The second prompt names its loop stage, so both parties know what does not belong in the session. It forces a plan restatement — the cheapest verification step in the whole book, catching stale context before any code exists. It pins the analysis to the frozen card rather than the model's judgment, converts silent failure into loud failure ("stop and say why"), and gives serendipity a labeled outlet instead of letting it contaminate the confirmatory run. The day-one prompt gets an answer; the contract-shaped

prompt gets an answer you can audit in five minutes, and the five-minute audit is what makes daily agent use compatible with signing your name to the results.

What Claude Code buys, in short, is enforcement surface: more places to install your method so that following it is the path of least resistance. The next chapter looks at the rest of the toolbox — including when the right answer is not Claude Code at all.

Chapter 6

opencode and the Google Stack

A workshop that taught only one vendor’s tool would be training, not methodology. This chapter covers the other three instruments on the bench — the open-source opencode, Google’s agent-first Antigravity platform, and Gemini CLI — and closes with selection rules, because in practice research groups run a mixed fleet and the interesting question is which tool gets which job.

6.1 opencode: The Open-Source Agent

opencode is a fully open-source agentic coding tool built by Anomaly, delivered primarily as a terminal TUI [1]. Architecturally it is the same loop as Chapter 4; three properties distinguish it for research use.

First, *provider-agnosticism*: opencode connects to any model provider you hold keys for — Anthropic, OpenAI, Google, or a local model server — so the tool choice stops being a model choice. For labs with data-governance constraints, pointing the same workflow at a locally hosted model is the difference between using agents and not using them; the sensitive-data pattern is to keep the workflow identical and swap the provider per project. Second, *inspectability*: the entire harness is readable source. When a methodology question turns on what the tool actually does — what enters context, how permissions are enforced — an open implementation lets you answer by reading rather than trusting. Third, the familiar anatomy is all present: `AGENTS.md` memory, plan mode (a keystroke toggles “suggest, don’t edit”), MCP support, sessions that can be shared as links for supervision or co-work, and a headless `opencode run` for pipelines [1]. Every lab in Part III lists its opencode equivalent in one line because the translation is nearly mechanical.

6.2 Google Antigravity: The Agent-First Platform

Antigravity, launched in November 2025, is Google’s “agentic development platform”: an IDE with a conventional AI-powered editor surface plus a *manager* surface — “a dedicated interface where you can spawn, orchestrate, and observe multiple agents working asynchronously” [8]. It is the one tool in this book that is not terminal-first, and it earns its chapter section with two ideas that matter methodologically.

The first is *artifacts*. Antigravity agents emit tangible intermediate deliverables — task lists, implementation plans, screenshots, browser recordings — and the human reviews and comments on these rather than watching a raw transcript [8]. That is Gate thinking built into the product: an implementation plan artifact is exactly the reviewable object Chapter 3’s Gate B wants, and a browser recording of the agent exercising your results dashboard is verification evidence, not vibes. The second is *browser control*: Antigravity agents can drive a browser to test what they built end-to-end. For applied research this closes a loop the terminal tools leave open — an agent that built your experiment-monitoring dashboard can also demonstrate, on video, that the dashboard shows the right numbers.

Antigravity is in public preview at no cost for individuals, runs on macOS, Windows, and Linux, and offers model optionality — Gemini 3 Pro at generous rate limits, plus Anthropic and open-weight models [8]. Its natural research role is the *supervision console*: several long-running

agent tasks (a refactor of the data pipeline, a batch of ablation runs, a documentation pass) observed from one place, each producing artifacts you can audit asynchronously.

6.3 Gemini CLI: The Free Workhorse

Gemini CLI is Google’s open-source (Apache-2.0) terminal agent — “an open-source AI agent that brings the power of Gemini directly into your terminal” — with GEMINI.md memory, MCP support, headless scripting, and built-in Google Search grounding [9]. Two facts drive its research niche. The free tier is generous (on the order of a thousand requests per day with a personal Google account), which makes it the default engine for high-volume, low-stakes batch work — first-pass literature triage, log summarization, format conversion — where per-request economics dominate. And search grounding means its answers can carry live web citations, useful for the sweep phase of Lab 1, with the usual rule that grounded snippets are leads to verify, never sources to cite.

6.4 Choosing: A Working Decision Rule

Figure 6.1 positions the four tools on the two axes that actually separate them in research use; everything else is converging anatomy.

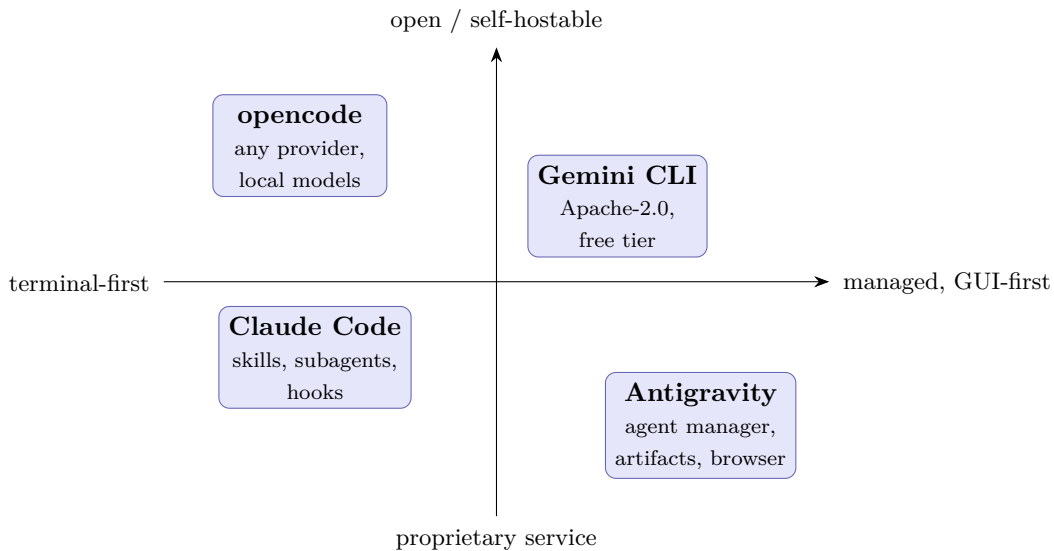


Figure 6.1: The toolchain positioned by interface philosophy (horizontal) and openness (vertical). Gemini CLI sits center-right of open: the client is Apache-2.0 while the models are a Google service. The quadrants, not the brands, are what to remember — new products will keep landing on this map, and the selection rules below read off coordinates, not names.

The decision rules the labs use: *sensitive data or audit-everything requirements* → opencode with a self-hosted or approved provider. *Deep workflow customization — contracts, subagents, hooks, skills* → Claude Code. *High-volume cheap batch work, or a zero-budget workshop seat* → Gemini CLI. *Parallel long-running tasks needing asynchronous supervision, or verification that must include a browser* → Antigravity. Most groups end up with a primary interactive tool plus Gemini CLI as the batch engine, which is exactly the fleet Part III assumes.

Data governance is a hard constraint, not a preference

Every proprietary-service tool in this chapter transmits your prompts — which in research means fragments of your data, hypotheses, and unpublished results — to a provider. Before any lab: know your institution’s policy, your data’s classification, and each provider’s retention terms. If the answer is “no external processing,” the open-source row of the map (opcode + local models) is your toolchain, at a real capability cost you should budget for honestly. Discovering this constraint *after* the data has left the building is not a recoverable error.

6.5 Budgeting: Tokens, Time, and Attention

Tool selection has a quieter counterpart: budgeting. Agentic sessions consume three currencies, and projects misprice them in a predictable order. *Tokens* are the visible cost — long sessions over large repositories are surprisingly expensive on metered API pricing and surprisingly cheap on subscription tiers and free tiers, which is why the fleet pattern routes batch work to Gemini CLI and keeps deep interactive work on a subscription seat. *Time* is the currency Chapter 2 warns about: measure your own loop, because perceived time savings are unreliable in exactly the settings where researchers work [5]. But *attention* is the binding constraint. Every delegated task returns as something to verify, and verification competes with your best hours. A researcher who spawns five parallel agent tasks has not multiplied their output by five; they have queued five review obligations, and the validation hierarchy of Chapter 10 prices each one.

The budgeting rule the labs assume: delegate until the review queue, not the token bill, is the bottleneck — then stop spawning and start verifying. Antigravity’s manager surface makes the queue visible, which is half its value; in terminal fleets the decisions file plays the same role if each entry names what is still awaiting review.

Fleet habits that keep methodology portable

- Maintain one contract file and symlink or copy it to `CLAUDE.md`, `AGENTS.md`, and `GEMINI.md` so every tool inherits the same rules.
- Keep prompts and plans in the repository, not in any tool’s private state — the repo is the portable layer.
- Re-run one lab’s pipeline when you swap tools; treat it as a calibration experiment, per Chapter 2’s measure-don’t-feel rule.

The bench is now stocked. Part III puts instruments to work.

Part III

The Practice: Workshop Labs

Chapter 7

Lab 1: Literature Review and Evidence Synthesis

The first laboratory builds the workflow every project starts with: turning a research question into a verified map of what is already known. It is deliberately the first lab because it exercises every skill the later ones assume — headless pipelines, structured outputs, and the verification ritual — on a task where mistakes are cheap and visible. Time budget in workshop form: 90 minutes.

7.1 The Shape of the Pipeline

A literature review with an agent is not “summarize these papers.” It is a pipeline with four distinct stages — sweep, extract, synthesize, verify — and the agent’s role changes at each one. Figure 7.1 shows the topology: many unreliable sources feeding one structured object, the claim–evidence matrix, which is the lab’s actual deliverable.

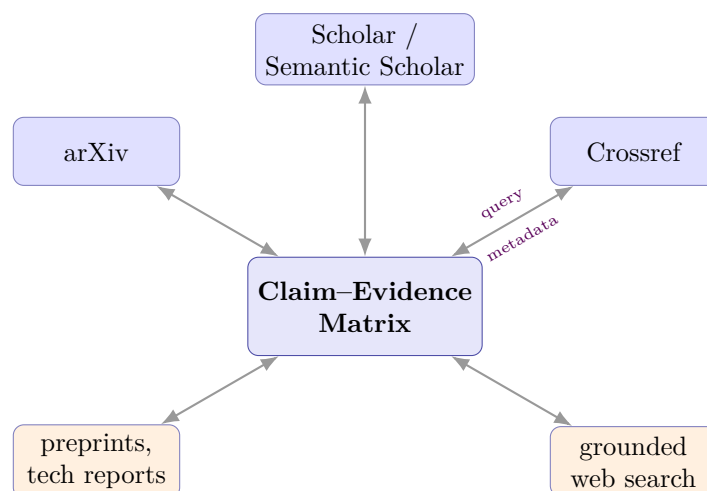


Figure 7.1: Evidence synthesis as hub-and-spoke. Blue spokes are scholarly indexes, orange spokes are lower-trust sources admitted only as leads. Every edge is bidirectional — queries out, records back — and nothing enters the hub matrix until the verify stage has fetched and read the primary source behind it.

7.2 Sweep and Extract

The sweep stage is mechanical and belongs to headless mode. The example below pulls candidate abstracts from arXiv’s public API; the same pattern works against any index your institution licenses (and Gemini CLI’s search grounding provides the general-web spoke).

```
# 1. Sweep: 50 newest arXiv abstracts matching the query
curl -s "http://export.arxiv.org/api/query?search_query=\
all:%22retrieval+augmented%22+AND+all:%22triage%22\
```

```
&max_results=50" > sweep.xml

# 2. Extract: one structured record per abstract
python split_entries.py sweep.xml corpus/ # one .txt each
for f in corpus/*.txt; do
  claude -p "From this abstract emit JSON:
    {id, claim, method, sample_size, effect,
    limitations, relevance_0_to_5}.
    Unknown fields -> null. Abstract:
    $(cat "$f")" >> records.jsonl
done
```

Listing 7.1: Sweep: pull candidates, then batch-extract structured records with a headless agent. The extraction prompt demands JSON so downstream stages are code, not copy-paste.

Why structured extraction beats summarization

A prose summary launders uncertainty: hedges, sample sizes, and non-findings vanish into fluent paragraphs. A fixed schema with nullable fields forces the model to say “unknown” where the abstract is silent — and the nulls are data. In workshop runs, the fraction of null **effect** fields is routinely the first interesting finding: it measures how much of the literature asserts without quantifying.

7.3 Synthesize: The Claim–Evidence Matrix

Synthesis happens interactively, with the records file in context. Ask the agent to pivot `records.jsonl` into a matrix whose rows are *claims* (not papers) and whose columns are supporting studies, contradicting studies, strongest effect size, and weakest link in the chain of evidence. Then interrogate it: which claims rest on a single group’s work, which have never been tested outside a benchmark, where do two well-cited results contradict. This is the diverge–converge pattern of Chapter 3 applied to reading: the agent proposes the map, the human decides what is actually load-bearing.

7.4 The Verification Ritual

Chapter 2’s warning is now operational: at this point the matrix is fluent and looks finished — and every cell of it is a claim awaiting verification. Nothing is citable yet.

Never cite what you have not fetched

Extraction ran on abstracts and search snippets: secondhand evidence. Models also confabulate plausible references when asked for sources, and abstracts routinely oversell what the paper’s own results section supports. The rule, with no exceptions for deadline pressure: a reference enters your bibliography only after you have fetched the primary source and confirmed it says what your sentence claims — including the direction, magnitude, and conditions of the effect.

The ritual itself is a per-claim loop, and it is agent-assisted but human-signed:

```
Here is the full text of the paper and a sentence I
intend to write citing it.
1. Quote the passage(s) that support my sentence.
2. State any way my sentence overstates or misstates
   the paper (population, magnitude, conditions).
3. List what the paper itself flags as limitations.
```

4. Verdict: cite as-is / weaken wording / do not cite.

Listing 7.2: The verification prompt, run per load-bearing claim, with the fetched paper (not the abstract) in context.

The verdicts are drafted by the agent; accepting each is the human’s act, logged in the decisions file. On the ten to fifteen load-bearing claims of a typical applied project, the whole ritual costs an afternoon — which is the price of a bibliography you can defend in review.

Lab 1 deliverables and exit criteria

- `records.jsonl` — structured extraction of the full sweep, nulls preserved.
- `matrix.md` — claim–evidence matrix, claims as rows, with contradiction and single-source flags.
- `references.bib` — *only* verified entries; every entry traceable to a fetched primary source.
- Decisions file lists each verification verdict you accepted or overrode.
- Exit question (Gate A feed): which one to three claims does your project now depend on, and what would refute each?

7.5 Adapting the Lab to Your Field

The pipeline’s stages are field-agnostic; the spokes and the schema are not, and adapting them is a fifteen-minute design task worth doing deliberately. The *sources* change first: engineering fields add standards bodies and patent databases (a patent sweep is the same pipeline with a different XML schema), biomedical work adds PubMed and trial registries, industry research adds internal wikis and incident postmortems — often the highest-value and least-indexed spoke, reachable by pointing the extraction loop at an exported directory instead of an API. The *schema* changes second, and it should encode your field’s skepticism: clinical extraction wants `study_design` and `n` as first-class fields; ML-systems work wants `hardware`, `baseline_tuned?`, and `code_available?`; policy-adjacent work wants `funder`. The verification ritual is the one component to adapt *least* — weaken it and the whole pipeline becomes a machine for laundering secondhand claims.

Two failure modes recur in adaptations. Teams over-widen the sweep (a thousand candidates feels rigorous, but synthesis quality is set by the extraction schema, not the corpus size — prefer 80 well-chosen entries with a sharp schema). And teams let the agent rank relevance unsupervised: calibrate it instead by hand-labeling ten records early and telling the agent where its scores disagreed with yours, which typically fixes the ranking for the rest of the corpus.

With the evidence base mapped, Lab 2 takes the surviving hypothesis into data.

Chapter 8

Lab 2: From Hypothesis to Experiment

The second laboratory is the core of the book: a pre-registered, agent-executed analysis, from hypothesis card to a result with defensible error bars. It is built so that the anti-patterns of Chapter 2 — specification search, silent metric swaps, fluent overstatement — are structurally difficult rather than merely discouraged. Workshop time budget: two hours with the provided dataset; the structure transfers to any tabular or experimental data you bring.

8.1 The Hypothesis Card

Lab 2 starts where Lab 1 ended: one surviving claim, now formalized into the card that will govern the whole analysis. The card is markdown, versioned, and short — if it exceeds a page, the hypothesis is not yet scoped.

```
# Hypothesis card -- H1 (frozen 2026-07-02, commit a3f1)

H1: Retrieval-augmented triage improves top-1 routing
accuracy by >= 5 points over the fine-tuned baseline
on tickets unseen during indexing.

Mechanism: errors concentrate in long-tail categories;
retrieval supplies category exemplars at inference.

Primary metric: top-1 accuracy, n = 4,000 held-out.
Test: two-sided permutation test on paired accuracy
      diff, 10,000 resamples; alpha = 0.01.
Also report: bootstrap 95% CI on the difference.
Exclusions: tickets with empty body (logged count).
Refutation: diff < 5 pts, or CI includes 0.
EXPLORATORY analyses go to explore/ -- never here.
```

Listing 8.1: analysis/PLAN.md — the hypothesis card. Committed before data contact; referenced as read-only by the agent contract of Listing 5.1.

Effect size

The magnitude of a difference or association in meaningful units (here: percentage points of accuracy), as distinct from its statistical significance. A tiny effect can be “significant” at scale, and applied decisions ride on magnitudes — which is why the card freezes a *minimum effect of interest* (5 points), not just an α level.

8.2 Executing the Frozen Plan

With the card committed, the session is deliberately boring: the agent loads the contract, restates the plan, and executes it — data loading, exclusions (with logged counts), metric computation, the permutation test, the bootstrap interval, and the figures. The researcher’s

job during execution is Chapter 5’s session rhythm: approve the plan restatement, then spot-check, not spectate. Two spot checks catch most silent failures: recompute one number by an independent path (a five-line pandas one-liner for the baseline accuracy), and ask the agent to print row counts at every filter step — count drift is the classic signature of a join or exclusion bug.

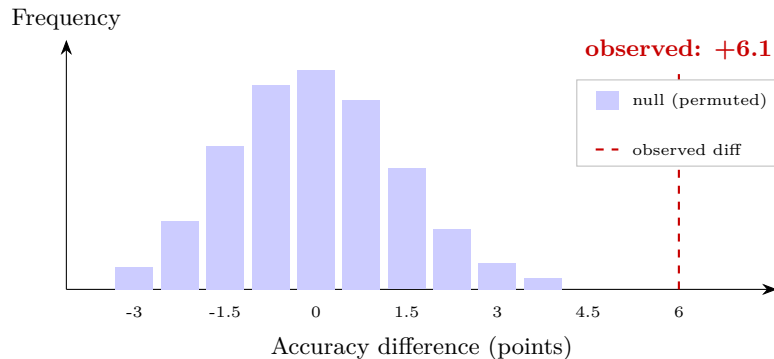


Figure 8.1: The pre-registered test, drawn the way it should be read: the pale bars are the null distribution of accuracy differences under 10,000 label permutations, and the dashed line is the observed difference. The question the figure answers is not “is there an effect?” but “how often does chance produce one this large?” — here, effectively never, and the observed +6.1 clears the card’s 5-point minimum effect of interest.

Why a permutation test as the lab’s default: it makes almost no distributional assumptions, its logic is explainable in one sentence (“shuffle the labels, see how big a difference chance makes”), and it is embarrassingly parallel — an agent writes a correct one in seconds and Figure 8.1 falls out of it for free.

8.3 Robustness: Attacking Your Own Result

A result that clears the pre-registered test earns the right to be attacked, not reported. The robustness stage is Chapter 3’s stage 7, and the agent makes formerly expensive checks cheap: rerun the pipeline with a different seed and confirm the headline is stable; recompute on ten bootstrap re-samples of the held-out set; rerun with the exclusion rule tightened and loosened one notch; split by the two or three covariates a skeptical reviewer would name (ticket length, category frequency, time window) and confirm the effect is not an artifact of one stratum.

Worked example: the robustness pass that saved a retraction

In a workshop run of this lab, H1 cleared the permutation test at +7.2 points. The stratified check then showed the entire effect concentrated in tickets shorter than 20 tokens — and a look at the retrieval index revealed those tickets were near-duplicates of indexed exemplars: leakage, not learning. The frozen plan was followed; it was the *attack stage* that caught the flaw. The corrected run (duplicates excluded, logged) came in at +5.4 — smaller, honest, and still past the minimum effect. The lesson participants remember: the agent found the leak in four minutes once asked; no one asks without a stage that demands it.

Anti-p-hacking guardrails, restated as habits

- The card is frozen in git before data contact; the agent contract forbids substitution (Gate B).
- Exactly one primary metric and test; everything else is labeled EXPLORATORY in filenames and output.
- Every filter logs its row count; count drift halts the run.
- The robustness pass is mandatory and its checks are chosen *before* seeing which ones the result would survive.
- Refutation is a reportable outcome: a killed hypothesis ships as a (short) result, which removes the incentive to torture it.

Lab 2's deliverables: the card, the analysis code, the headline number with its CI, the robustness table, and a decisions file that lets a stranger reconstruct what was pre-registered versus explored. What it does *not* yet have is a form anyone else can run. Making the result portable — reproducible by a reviewer, a colleague, or you in six months — is Lab 3.

Chapter 9

Lab 3: Reproducibility and Reporting

The reproducibility statistics of Chapter 2 — half of surveyed scientists unable to reproduce their own experiments [3] — are not a moral failing; they are what happens when reproduction is expensive. The third laboratory makes it cheap: the deliverable is a repository where one command re-derives every number and figure in the report, plus a report whose prose survived an adversarial review. This is also where agent leverage is largest, because packaging work is exactly the well-specified, mechanically checkable labor agents do best.

9.1 The Reproducibility Stack

Reproducibility is layered, and each layer answers one specific way results rot. Figure 9.1 names the layers; the lab builds them bottom-up.

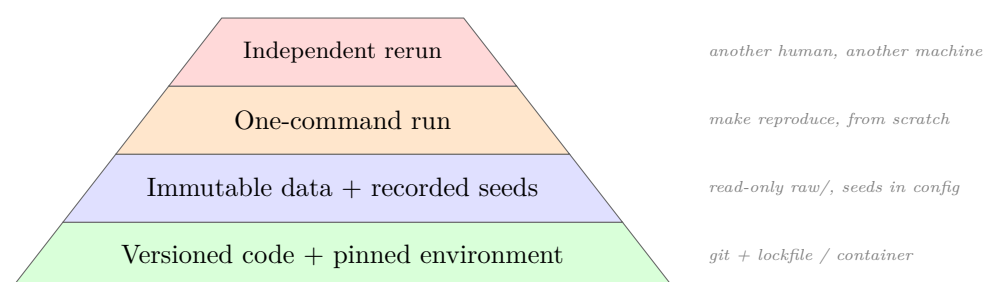


Figure 9.1: The reproducibility stack. Each layer assumes the ones below it: a one-command run means nothing if the environment drifts, and an independent rerun is the only layer that tests the instructions rather than the author’s memory. The lab’s exit criterion is the top layer — a workshop partner reproduces your headline number on their machine.

The agent’s role is to build the bottom three layers from the existing Lab 2 repository: freeze the environment (lockfile and a container image), verify `data/raw/` is read-only and every random seed flows from one config value, and write the orchestration so the whole pipeline re-derives from raw data:

```
# Containerfile: pinned base + locked deps (agent-drafted)
# FROM python:3.12-slim
# COPY requirements.lock .
# RUN pip install --no-cache-dir -r requirements.lock

make reproduce # raw data -> results/ (fresh container)
make verify    # recompute report.md numbers; diff; fail loudly

# the whole loop, from a clean clone:
git clone <repo> && cd <repo> && make reproduce verify
```

Listing 9.1: The one-command contract, and the check that keeps it honest. The verify step recomputes every reported number and diffs against the report’s values — run it in CI or nightly via headless mode (Listing 5.2).

Why agents change the reproducibility economics

The reason most projects skip these layers is that packaging is a day of unglamorous work performed after the intellectual reward has already been collected. An agent compresses that day to under an hour — drafting the Containerfile, chasing down unpinned imports, wiring the Makefile, and (most valuably) *testing* the clean clone path in a scratch directory until it actually works. The excuse is gone; what remains is whether your workflow demands it.

9.2 Reporting: The Agent as Critic, Never Author

For the report itself, the division of labor inverts. In analysis the agent executed and the human judged; in writing the human authors and the agent attacks. The reasons are Chapter 2's: fluent generated prose launders overstatement past its own author [16], and the person who signs the report must be able to defend every sentence without a transcript. The productive agent roles are reviewer and auditor:

You are a hostile but fair reviewer. For this draft:

1. List every claim not supported by a number in results/ -- quote the sentence.
2. Find sentences that overstate the evidence (magnitude, causality, generality) and rewrite each at the strength the data supports.
3. Name the three weakest points a referee will attack, and what analysis or caveat would answer each.
4. Check every citation: does the cited work say what the sentence claims? Flag any you cannot confirm from the sources in refs/.

Listing 9.2: The adversarial review prompt, run on your drafted report with the repository in context.

Run it twice — once when the draft is complete, once after revisions — and log both verdicts. The second pass is routinely where residual overclaiming is caught, because authors weaken sentences the reviewer flagged and unconsciously strengthen others in compensation.

Release checklist (Gate C, in full)

- make reproduce && make verify passes from a clean clone, in the container, on a machine that is not yours.
- An independent human reproduced the headline number (workshop pairs do this for each other).
- Report states the pre-registered claim, the exploratory findings, and the robustness results *separately*, with the hypothesis card linked by commit hash.
- Adversarial review run twice; both transcripts kept.
- Every citation verified per Lab 1's ritual; the bibliography contains nothing you have not read.
- Data, code, and container published to whatever venue your field and policies allow; where they cannot be, the report says so and why.

A note on scope: layers, not heroics. A two-day internal analysis does not need a container registry and a DOI — but it still gets the bottom two layers (versioned code, immutable data,

seeds), because those cost minutes and are the difference between an auditable result and an anecdote. The stack is a menu you cut from the top, never from the bottom.

The three labs together instantiate the full eight-stage loop: Lab 1 feeds Gates A, Lab 2 runs stages 4–7, Lab 3 closes Gate C. What remains is what you take home when the workshop laptop closes — the field manual.

Chapter 10

The Researcher's Field Manual

Workshops end; Tuesday afternoons with a deadline do not. This closing chapter collects the prompts, checklists, and decision rules from the whole book into the form they will actually be used in: short, copyable, and annotated with just enough rationale to survive being separated from the chapters that justify them.

10.1 The Validation Hierarchy

One picture organizes everything the book asks you to do after an agent produces something. Not all outputs deserve the same scrutiny; Figure 10.1 matches the check to the stakes.

Verification effort rises with stakes; volume falls.

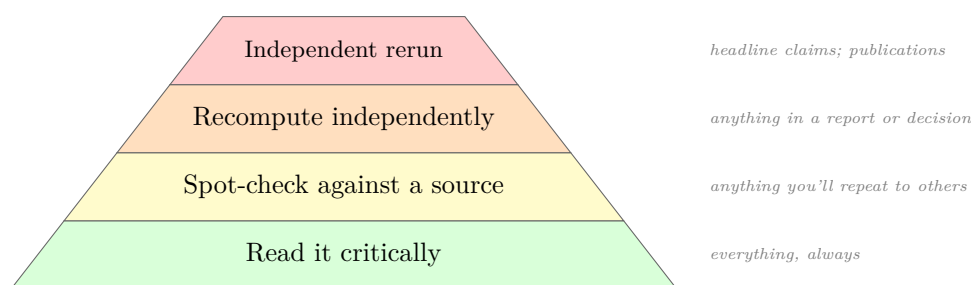


Figure 10.1: The validation hierarchy. Every agent output gets the bottom layer; each layer above it applies to a smaller, more consequential subset. The practical rule: before using any output, name the layer it needs — “which layer?” is a faster and more honest question than “do I trust this?”

10.2 The Prompt Library

Four prompts cover the recurring situations. They are short by design — long prompts do not survive contact with a deadline — and each names the chapter carrying its rationale.

```
Here is a candidate hypothesis: <card draft>.
1. Argue it is false: strongest mechanism-level case.
2. Argue it is untestable as scoped: what's underspecified?
3. Name the confound most likely to produce a false
   positive, and the cheapest design change that
   controls it.
Do not soften. I am deciding whether to spend a month.
```

Listing 10.1: The hypothesis critic (Ch. 3, Gate A) — run on every candidate before scoring the rubric.

```
Audit the analysis in <path>:
1. Reconstruct what was actually computed, step by
   step, from the code -- not from comments or names.
2. List every researcher degree of freedom exercised
   (filters, metric choices, windows) and whether each
   is in the pre-registration.
```

3. Print row counts at each filter; flag any drift.
4. What is the single most fragile step?

Listing 10.2: The statistical auditor (Ch. 8) — run after any analysis, before believing it.

```
In a fresh clone of this repo, attempt:
make reproduce && make verify.
Report every failure verbatim, fix nothing.
Then list each reported number in report.md that the
verify step does not cover.
```

Listing 10.3: The repro auditor (Ch. 9) — run in a scratch clone, not your working copy.

The fourth is Lab 1’s verification prompt (Listing 7.2), unchanged, and the adversarial reviewer (Listing 9.2) completes the set for writing days.

10.3 The Checklists

Before starting an agent session

- Which loop stage (Ch. 3) is this session? One session, one stage.
- Is the contract file current — does it reference today’s frozen plan?
- What will I verify before acting on the output, and at which layer of Figure 10.1?
- If this session’s result would be exciting, what check would kill it? Schedule that check now.

Before shipping anything agent-assisted

- I can explain every claim without opening a transcript.
- Pre-registered vs. exploratory is labeled in the text, not just in my memory.
- Citations: fetched, read, verified — no exceptions survived the deadline.
- The adversarial review ran; its worst finding is fixed or disclosed.
- Provenance: transcript kept, commits at stage boundaries, decisions file current.

10.4 Situations to Rules

Table 10.1: Situation-to-rule quick reference, with the chapter carrying the full argument.

Situation	Rule	Ch.
Agent proposes an exciting result	Attack it before reporting it	8
Idea feels novel and fluent	Score the rubric; run the critic	2–3
Choosing a tool for a new project	Read the quadrant, not the brand	6
Sensitive data in play	Open-source tool + approved provider	6
Tempted to tweak the metric mid-analysis	It’s exploratory now; label it	8
“The agent said the paper says...”	Fetch it or don’t cite it	7
Workflow feels faster	Feeling is inadmissible; measure	2
Result about to ship	Clean-clone reproduce + verify	9

10.5 Objections, Answered Briefly

“This process overhead will eat the productivity gains.” The overhead is one card, three gates, and an afternoon of verification per project — against the documented alternative: confident self-assessed speedups that measure as slowdowns [5], and quality losses precisely where the tool feels strongest [7]. The method is not a tax on the gains; it is what makes them real.

“Models are improving; next year this book is obsolete.” The products will change — that is why Chapter 4 taught anatomy and Chapter 6 taught quadrants rather than key-bindings. The methodology is model-agnostic by construction: better models make the diverge stages richer and the gates *more* valuable, because the cost of unverified fluency rises with how convincing it is.

“My field’s real bottleneck is experiments, not analysis.” Then the agent’s leverage concentrates in Labs 1 and 3 — evidence synthesis and reproducibility — and in the instrument-control frontier that Coscientist previewed [6]. The loop does not require that every stage accelerate equally; it requires that no stage skip its gate.

10.6 Start Monday

A minimal on-ramp, in ascending commitment. Monday: create the contract file in your current project and put one hard rule in it. This week: run Lab 1’s verification ritual on the three references your current draft leans on hardest. This month: take one live hypothesis through the full loop — card, frozen plan, attack stage, clean-clone reproduce. This quarter: run the METR calibration on yourself — pick a recurring task, measure a week with the agent and a week without, and believe the clock over the feeling [5]. The tools will keep getting faster. The method is what decides whether you get faster *research* — let the agent carry the work; keep the judgment.

Chapter A

Workshop Setup Guide

Complete this appendix *before* the workshop — installs and authentication are the classic first-twenty-minutes sink, and the labs assume a working seat. Budget 30–45 minutes plus download time. Everything here is also the setup path for a reader working through the labs alone.

A.1 Prerequisites

All platforms need three things the labs treat as given: a terminal with `git` (on Windows, install Git for Windows so the tools get a POSIX shell), Python 3.11+ with `venv`, and `make` (ships with Xcode tools on macOS and `build-essential` on Debian/Ubuntu; on Windows it arrives with Git Bash or via `winget install GnuWin32.Make`). Lab 3 additionally wants a container engine — Docker Desktop or Podman — but has a degraded no-container path if your machine or policy forbids one.

A.2 Installing the Tools

You need Claude Code working and at least one of the other three; install all four if your seat allows, since Chapter 6’s selection rules are easier to internalize with the tools side by side.

```
# --- Claude Code (needs Claude subscription or API key)
curl -fsSL https://claude.ai/install.sh | bash
# Windows PowerShell:
# irm https://claude.ai/install.ps1 | iex
claude --version

# --- opencode (open source; bring any provider key)
curl -fsSL https://opencode.ai/install | bash
opencode --version

# --- Gemini CLI (free tier with a Google account)
npm install -g @google/gemini-cli
gemini --version

# --- Google Antigravity (GUI platform, not a CLI)
# download the installer for your OS from
# https://antigravity.google/ and sign in
```

Listing A.1: Installs, one block per tool. Run the block for each tool you are seating; each ends with the version check that confirms the install landed.

Authentication is per-tool and interactive: run `claude` once in any directory and follow the login flow; run `opencode auth login` and paste the provider key you intend to use; run `gemini` once and complete the Google sign-in. Corporate networks that intercept TLS are the most common failure here — if a login loop hangs, try a personal network before debugging anything else.

A.3 The Practice Repository

The labs run against a small ticket-triage dataset and a skeleton repo you create once:

```
mkdir triage-lab && cd triage-lab && git init
mkdir -p data/raw results analysis explore corpus
python -m venv .venv && source .venv/bin/activate
pip install pandas numpy matplotlib
# fetch the workshop dataset (or substitute your own CSV)
# -> data/raw/tickets.csv, then protect it:
chmod -R a-w data/raw
printf '# CLAUDE.md\n(see Listing 5.1 in the book)\n' \
  > CLAUDE.md
cp CLAUDE.md AGENTS.md && cp CLAUDE.md GEMINI.md
git add -A && git commit -m "workshop skeleton"
```

Listing A.2: Creating the workshop repository. The layout encodes the permission discipline of Chapter 4: raw data read-only, outputs segregated, contract at the root.

A.4 Seat Check

Your seat is ready when all four lines below behave as described — verify this the day before, not in the opening session:

```
git -C triage-lab status          # clean tree
echo "2+2?" | claude -p "answer" # model reachable
claude -p "read CLAUDE.md and quote its first rule" \
  # proves file access + contract pickup
python -c "import pandas"        # analysis env OK
```

Listing A.3: The seat check. Total runtime under two minutes; any failure points at exactly one layer of the stack.

If only the second line fails, the problem is authentication or network, not the install. If the third fails, you are running outside the repo directory — the tools scope file access to where they are launched, which is a feature you want.

Facilitator notes (running this as a workshop)

- Send this appendix out a week ahead; open with the seat check, not with slides.
- Have two or three Gemini CLI free-tier seats ready as fallbacks for participants whose primary tool fails authentication on site.
- Pair seats for Lab 3: the independent-rerun exit criterion (Figure 9.1) needs a second machine.
- Timebox: Lab 1 $\approx$ 90 min, Lab 2 $\approx$ 120 min, Lab 3 $\approx$ 90 min, with Chapter 3 taught before Lab 1 and Chapter 2's evidence woven into the debriefs.

Chapter B

Templates

Copy-ready versions of the four artifacts every project in this book creates. They are deliberately minimal: each field earns its place by blocking a documented failure mode, and anything you add should meet the same bar.

B.1 The Contract File

The full research contract, generalizing Listing 5.1. Save as `CLAUDE.md` and copy to `AGENTS.md` / `GEMINI.md` per Chapter 6's fleet habit.

```
# CLAUDE.md -- research contract

## Project
<one-paragraph aim>. Pre-registration: analysis/PLAN.md
(frozen at commit <hash>).

## Hard rules
- analysis/PLAN.md is READ-ONLY: execute exactly; if a
  step is impossible, STOP and report.
- data/raw/ is read-only; outputs go to results/;
  exploratory work goes to explore/ and is labeled
  EXPLORATORY in every output it produces.
- Every numeric claim in a summary names the script
  and input that produced it.
- Never fabricate a citation; sources come only from
  refs/ (fetched full texts).

## Environment
- setup: <venv/conda activation>
- tests: <command> - reproduce: <command>

## Session discipline
- One loop stage per session; commit at stage end.
- Append 5 lines to DECISIONS.md before ending.
```

Listing B.1: Contract-file template. Replace angle-bracketed fields; delete nothing without deciding what replaces its function.

B.2 The Hypothesis Card

The frozen plan of Listing 8.1, as a fill-in skeleton:

```
# Hypothesis card -- <ID> (frozen <date>, commit <hash>)

H: <falsifiable claim with direction and minimum
  effect of interest, on a named population/system>
Mechanism: <why this should be true>
Decision at stake: <who does what differently>
```

```
Primary metric: <one metric, one dataset, n>
Test: <one test, alpha, sidedness>
Also report: <CI / effect size, no new claims>
Exclusions: <rules fixed now; counts to be logged>
Refutation: <the observation that kills H>
```

Listing B.2: Hypothesis-card template (analysis/PLAN.md). Freeze by committing; amendments are new commits with reasons, never edits in place.

B.3 The Decisions File

```
## <date> <session tag>
Stage: <loop stage 1-8>
Did: <what the session produced, one line>
Decided: <the human judgment made, one line>
Because: <the reason, one line>
Parked: <anomalies/ideas deferred, with where>
```

Listing B.3: Decisions-file entry format (DECISIONS.md) — the one provenance artifact only a human can write.

B.4 The Reproduction Makefile

```
.PHONY: reproduce verify test

reproduce:
    python analysis/run.py --config analysis/config.yaml
    python analysis/figures.py

verify: reproduce
    python analysis/check_report.py report.md results/
    @echo "verify: all reported numbers reproduced"

test:
    pytest analysis/tests -q
```

Listing B.4: Makefile skeleton implementing Lab 3’s one-command contract. `verify` fails loudly if any reported number drifts.

Together with the prompt library of Chapter 10, these four templates are the entire written apparatus of the methodology. If your project has them current — contract, card, decisions, Makefile — you are running the loop; if any is stale, that is the maintenance to do before the next session.

Further Reading: A Map of the Evidence

This book’s argument rests on a compact set of load-bearing sources. This map groups them by the question they answer, with a sentence on what each contributes — and, in the spirit of Lab 1, you should fetch and read the ones your own work will lean on rather than trusting these summaries.

What has AI actually contributed to science?

Wang et al. [21] is the broad, sober review of AI across the research pipeline and the right starting point. Jumper et al. [13] is the canonical existence proof that a learned model can solve a scientific grand challenge. Romera-Paredes et al. [19] shows the search-plus-verifier pattern producing new mathematics, and Boiko et al. [6] shows an agent operating real laboratory instruments. For the run-the-whole-pipeline frontier, read Lu et al. [15] and its workshop-reviewed successor Yamada et al. [22] critically — noting how much verification still leans on humans — and Gottweis et al. [10] for the hypothesis-tournament architecture this book’s diverge–converge stage borrows.

Should I trust AI-assisted work — including my own?

Four studies calibrate expectations. Peng et al. [18] is the optimistic controlled result on a well-specified task; Dell’Acqua et al. [7] maps the jagged frontier where gains flip to quality losses; Lee et al. [14] documents confidence in AI crowding out critical thinking; and Becker et al. [5] is the essential corrective — experts measurably slowed while feeling accelerated. Si et al. [20] is the cleanest evidence on AI-generated research ideas (more novel, less feasible), and Messeri and Crockett [16] names the systemic risk: producing more while understanding less. Bastani et al. [4] carries the design lesson — guardrails flip the sign of the effect — that this book’s gates operationalize.

Why all the methodology?

Ioannidis [11] is the four-page argument that selection plus flexibility makes most positive findings false; agents multiply the flexibility. Open Science Collaboration [17] and Baker [3] supply the empirical base rates. Together they are the reason Chapters 3 and 8 freeze plans before data contact.

Where do the tools themselves live?

Primary documentation, all current as of this writing: Anthropic [2] for Claude Code, Anomaly [1] for opencode, Google [8] for the Antigravity launch and concepts, and Google [9] for Gemini CLI. Jimenez et al. [12] explains the benchmark behind agent capability claims — read its evaluation setup once and you will read tool marketing differently.

A closing calibration note: the agentic-tools literature is young — several key results are preprints, the products change quarterly, and replications are pending. The durable content here is the method. Where this book and better evidence disagree, follow the evidence, and update your contract file accordingly.

References

- [1] Anomaly. opencode: The open-source AI coding agent — documentation. <https://opencode.ai/docs>, 2026. Accessed July 2026.
- [2] Anthropic. Claude code documentation. <https://code.claude.com/docs>, 2026. Accessed July 2026.
- [3] Monya Baker. 1,500 scientists lift the lid on reproducibility. *Nature*, 533:452–454, 2016. doi: 10.1038/533452a.
- [4] Hamsa Bastani, Osbert Bastani, Alp Sungu, Haosen Ge, Özge Kabakcı, and Rei Mariman. Generative AI without guardrails can harm learning: Evidence from high school mathematics. *Proceedings of the National Academy of Sciences*, 122(26), 2025. doi: 10.1073/pnas.2422633122.
- [5] Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein. Measuring the impact of early-2025 AI on experienced open-source developer productivity. arXiv preprint arXiv:2507.09089, 2025.
- [6] Daniil A. Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023. doi: 10.1038/s41586-023-06792-0.
- [7] Fabrizio Dell’Acqua, Edward McFowland III, Ethan Mollick, Hila Lifshitz-Assaf, Katherine C. Kellogg, Saran Rajendran, Lisa Kraye, François Candelon, and Karim R. Lakhani. Navigating the jagged technological frontier: Field experimental evidence of the effects of AI on knowledge worker productivity and quality. Harvard Business School Working Paper 24-013, 2023.
- [8] Google. Build with Google Antigravity, our new agentic development platform. Google Developers Blog, <https://developers.googleblog.com/build-with-google-antigravity-our-new-agentic-development-platform/>, 2025. November 20, 2025; accessed July 2026.
- [9] Google. Gemini CLI: An open-source AI agent for your terminal. <https://github.com/google-gemini/gemini-cli>, 2026. Apache-2.0; accessed July 2026.
- [10] Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Anil Palepu, Petar Sirkovic, Artiom Myaskovsky, Felix Weissenberger, Keran Rong, Ryutaro Tanno, et al. Towards an AI co-scientist. arXiv preprint arXiv:2502.18864, 2025.
- [11] John P. A. Ioannidis. Why most published research findings are false. *PLoS Medicine*, 2(8):e124, 2005. doi: 10.1371/journal.pmed.0020124.
- [12] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.

- [13] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596:583–589, 2021. doi: 10.1038/s41586-021-03819-2.
- [14] Hao-Ping Lee, Advait Sarkar, Lev Tankelevitch, Ian Drosos, Sean Rintel, Richard Banks, and Nicholas Wilson. The impact of generative AI on critical thinking: Self-reported reductions in cognitive effort and confidence effects from a survey of knowledge workers. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2025. doi: 10.1145/3706598.3713778.
- [15] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist: Towards fully automated open-ended scientific discovery. arXiv preprint arXiv:2408.06292, 2024.
- [16] Lisa Messeri and M. J. Crockett. Artificial intelligence and illusions of understanding in scientific research. *Nature*, 627:49–58, 2024. doi: 10.1038/s41586-024-07146-0.
- [17] Open Science Collaboration. Estimating the reproducibility of psychological science. *Science*, 349(6251):aac4716, 2015. doi: 10.1126/science.aac4716.
- [18] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. The impact of AI on developer productivity: Evidence from GitHub Copilot. arXiv preprint arXiv:2302.06590, 2023.
- [19] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024. doi: 10.1038/s41586-023-06924-6.
- [20] Chenglei Si, Diyi Yang, and Tatsunori Hashimoto. Can LLMs generate novel research ideas? A large-scale human study with 100+ NLP researchers. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025. arXiv:2409.04109.
- [21] Hanchen Wang, Tianfan Fu, Yuanqi Du, Wenhao Gao, Kexin Huang, Ziming Liu, Payal Chandak, Shengchao Liu, Peter Van Katwyk, Andreea Deac, et al. Scientific discovery in the age of artificial intelligence. *Nature*, 620:47–60, 2023. doi: 10.1038/s41586-023-06221-2.
- [22] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist-v2: Workshop-level automated scientific discovery via agentic tree search. arXiv preprint arXiv:2504.08066, 2025.